OBJECTS IN JAVASCRIPT / Short introduction only / Part 1

JavaScript is an *object-oriented programming language*[1]. To develop applications in JavaScript effectively you need to understand how to work with objects.

Objects can encapsulate both ***data*** (several different data items in a single object) and ***behavior***. The behavior of an object is defined by defining one or more methods of the object. In JavaScript, a ***method*** is a JavaScript function that is invoked through an object.

1 Literals

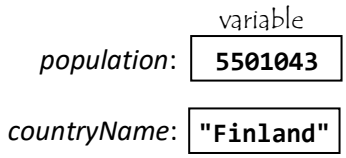
Fixed values in the program code are called ***literals***.

`12 3.14`	number literals
`"Hello!" "123" '"id": 100'`	string literals
`true false`	boolean literals
`["bird", "cat", "dog"]`	an array initialization literal
`{name: "Finland", population: 5501043}`	an ***object literal***
`[ {name: "Finland", population: 5501043},` `{name: "Sweden", population: 9747355} ]`	an array initialization literal that includes two ***object literals***

2 Primitive types and reference types

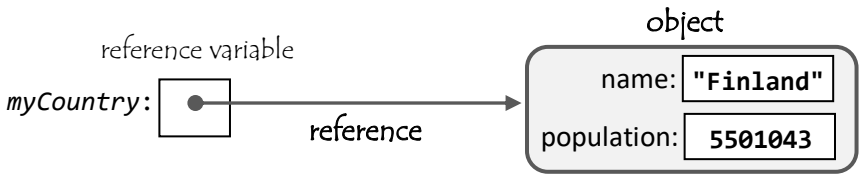
A ***primitive value*** is data that is not an object. In JavaScript, primitive data types for storing data are *number*, *string* and *boolean*. Examples of primitive values are **`12`**, **`"Hello!"`** and **`true`**.

```
var population = 5501043;
var countryName = "Finland";
```



A ***reference type value*** is a reference to an instance of the type, the latter known as an ***object.*** If the variable contains a reference to an object, we call the variable as ***reference variable***.

```
var myCountry = {name: "Finland", population: 5501043};
```



The simplest possible object is just a ***collection of named values***. In JavaScript, these named values are referred to as ***properties*** of the object. In the above, an object literal is used for creating an object. The object includes two properties (*name* and *population*). A reference to the object is assigned to *myCountry*. The properties of the object can be accessed as follows:

console.log("The population of " + *myCountry*.name + " is " + *myCountry*.population);

[1] Examples of other popular object-oriented programming languages are Java, C#, C++, and Python.
NB! These programming languages are *quite well-designed*, ***strictly typed,*** and objects are ***class-based.***
JavaScript is *quite poorly designed*, ***loosely-typed,*** and JavaScript objects are ***protype-based***.

3 Using object literals

An ***object literal*** consists of a comma-separated list of colon-separated property/value pairs, enclosed in curly braces.

```
var myCountry = {name: "Finland", population: 5501043};
```

To refer to a property of an object, you refer to the object, followed by a period and the name of the property.

```
console.log("The population of " +
    myCountry.name + " is " + myCountry.population);
```

The code above prints the following to the console window:

```
The population of Finland is 5501043
```

Object literals can also be nested. For example:

```
var country =  {
      name: "Finland",
      population: 5501043,
      capital: {name: "Helsinki", population: 634509}
};
```

Properties of nested objects can be accessed by chaining dot references together. In the example below, we write *country.***capital.name** to get the name of the capital city.

console.log("The capital of " + *country.***name** + " is " + *country.***capital.name**);

The code above prints the following to the console window:

```
The capital of Finland is Helsinki
```

Of course, we can assign new values to properties. For example:

```
country.capital.population = 634525;
```

You can include literals in an array initializer as follows:

```
var countryArray = [
    {name: "Finland", population: 5501043},
    {name: "Sweden", population: 9747355},
    {name: "Norway", population: 5165802}
];

for (var i = 0; i < countryArray.length; i++) {
    console.log(countryArray[i].name);
}
```

The code above prints the following to the console window:

```
Finland
Sweden
Norway
```

Here is one more example of accessing the array :

```
var countryArray = [
    {name: "Finland", population: 5501043},
    {name: "Sweden", population: 9747355},
    {name: "Norway", population: 5165802}
];

countryArray.sort(function(a, b) { return a.population - b.population; });

console.log("The lowest population (" +
    countryArray[0].population + ") is in " + countryArray[0].name);
```

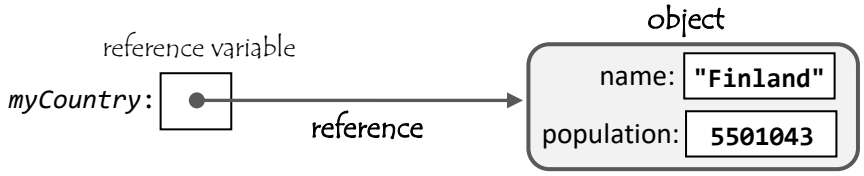
The code above prints the following to the console window:

```
The lowest population (5165802) is in Norway
```

4 In JavaScript, NO VARIABLE CAN EVER HOLD AN OBJECT!

In JavaScript, an object itself is ***never*** stored in a variable. Instead, a variable can store a ***reference to an object***. If a variable contains a reference to an object, we can call the variable ***reference variable***.

```
var myCountry = {name: "Finland", population: 5501043};
```

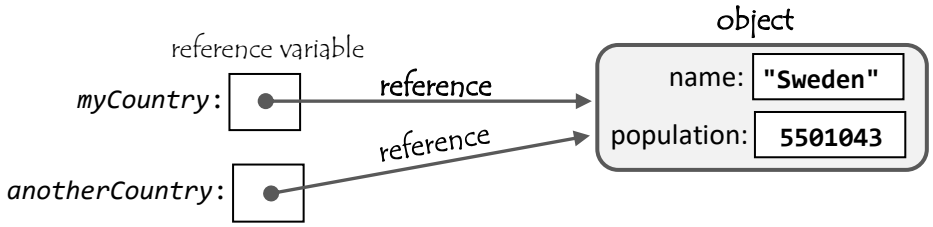


In the above, we assign something to the variable *myCountry*. Instead of a primitive value there is an *object literal* on the right-hand side of the assignment operator. Therefore, the system first creates a new object based on the object literal and then it assigns a reference to the object to the variable.

Let's see what happens if we **copy a value from a *reference variable* to another *reference variable*.**

```
var myCountry = {name: "Finland", population: 5501043};
var anotherCountry = myCountry;
anotherCountry.name = "Sweden";
console.log(myCountry.name);
```

The code above prints "Sweden" to the console window. **Why?** See the diagram below...



In the assignment ***anotherCountry = myCountry*** a value from one reference variable is copied to another reference variable. In the case of a ***reference variable***, the copied value is a ***reference*** (the referenced object is *not* copied). This is called ***reference copy***.

5 JSON (JavaScript Object Notation)

See also http://www.json.org

JSON (JavaScript Object Notation) is a language-independent lightweight open-standard data-interchange format that uses human-readable text to transmit data objects.

- Nowadays, JSON is the most common data-interchange format used for asynchronous browser/server communication.

The JSON format is based on a subset of JavaScript. A ***JSON string*** is similar to a JavaScript object literal. The main difference is that *property names must be enclosed in quotes* in JSON.

The JSON.**stringify** method converts a JavaScript object (or any value) to a JSON string.

```
var myCountry = {
    name: "Finland",
    population: 5501043,
    capital: {name: "Helsinki", population: 634509}
};

var jsonString = JSON.stringify(myCountry);

console.log(jsonString);
```

The code above prints the following to the console window:

```
{"name": "Finland", "population": 5501043,"capital": {"name": "Helsinki", "population": 634509}}
```

The JSON.**parse** method parses a JSON string and constructs a JavaScript object (or value) described by the JSON string.

```
var countryObject = JSON.parse(jsonString);

console.log("The capital of " + countryObject.name + " is " + countryObject.capital.name);
```

More advanced JSON example

The JSON string below describes a *list of three countries* and the *region where they are located*. Each country is described with the name of the country and population.

Typically, a server sends this kind of JSON string to the JavaScript program. To keep things simple in this example, the JSON string is defined below as an ordinary string literal.

```
var jsonString =
    '{"countries":[{"name": "Finland","population": 5501043},
     {"name": "Sweden", "population": 9747355},
     {"name": "Norway", "population": 5165802}], "region": "Europe"}';

var obj = JSON.parse(jsonString);

console.log("The countries below are located in " + obj.region);

for (var i = 0; i < obj.countries.length; i++) {
    console.log(obj.countries[i].name + ", population: " +
                obj.countries[i].population);
}
```

The code above prints the following to the console window:

```
The countries below are located in Europe
Finland, population 5501043
Sweden, population 9747355
Norway, population 5165802
```